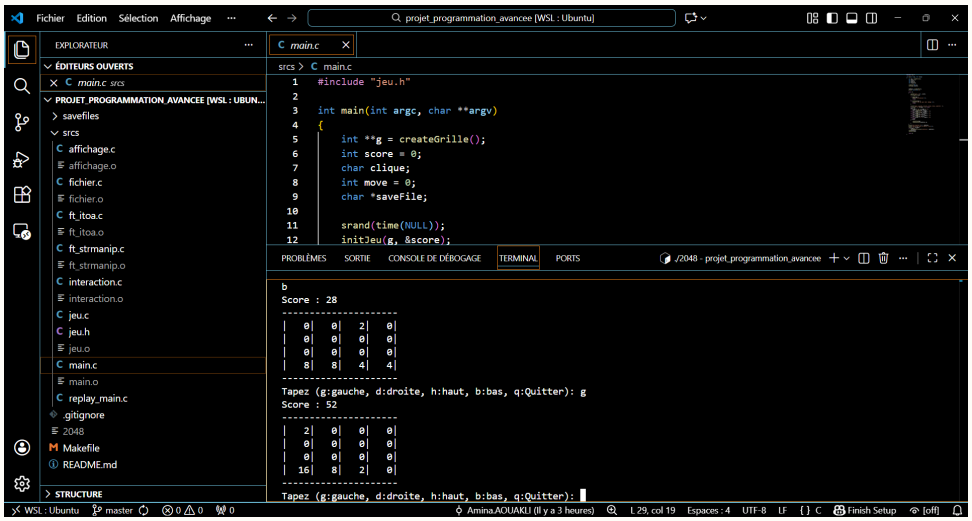
Rapport de Projet Jeu 2048 en C

22 décembre 2025



— **Membres du groupe :** Amina Aouakli et Quentin Moreau

— **Dépôt GitLab du groupe :** https://gitlab.isima.fr/projetProgrammationAvancee.git

Table des matières

1 Organisation du Travail **2**

2 Description détaillée du contenu du dossier `srcs` **2**

2.1 Fichier jeu.c . . . 2
2.1.1 createGrille . . . 2
2.1.2 liberGrille . . . 2
2.1.3 ajouterTuile . . . 2
2.1.4 calcul . . . 2
2.1.5 movGauche, movDroite, enHaut, enBas . . . 3
2.1.6 estTermine . . . 3
2.1.7 gagner . . . 3
2.2 Fichier fichier.c . . . 3
2.2.1 saveStateToFile . . . 3
2.2.2 initSaveFile . . . 3
2.2.3 outputLine . . . 3
2.2.4 replayGame . . . 4
2.2.5 lineToGrille . . . 4
2.2.6 recupGrilleEnCours . . . 4
2.3 Fichier ft_itoa.c : . . . 4
2.3.1 ft_taille . . . 4
2.3.2 fill . . . 4
2.3.3 ft_itoa . . . 4
2.3.4 ft_atoi . . . 4
2.4 Fichier ft_strmanip.c : . . . 5
2.4.1 ft_strlen . . . 5
2.4.2 ft_strappend . . . 5
2.4.3 ft_strjoin . . . 5
2.4.4 ft_str_replace . . . 5
2.5 Fichier affichage.c : . . . 5
2.5.1 tailleInt . . . 5
2.5.2 affichageGrille . . . 5
2.6 Fichier replay_main.c . . . 5
2.6.1 replayMode . . . 5
2.6.2 main . . . 6
2.7 Fichier main.c . . . 6
2.7.1 main (Mode de jeu interactif) . . . 6

3 le Makefile **6**

3.1 Configuration technique . . . 6
3.2 Commandes principales . . . 6

4 Fin **6**

4.1 Exécuter . . . 6

1 Organisation du Travail

Pour bien commencer, nous avons pris le temps de lire attentivement l'énoncé afin de comprendre les objectifs et les exigences du projet. Ensuite, nous avons planifié notre travail et réparti les fonctionnalités entre différents fichiers et dossiers essentiels pour structurer le projet de manière efficace :

- **Dossier** `srcs` : Contient l'ensemble des fichiers sources du projet :
  - `jeu.c` : Regroupe toute la logique du jeu (initialisation de la grille, gestion des déplacements, fusions des tuiles et calcul ...).
  - `jeu.h` : Fichier d'en-tête (header) contenant les définitions des structures, les prototypes de fonctions et les constantes partagées par l'ensemble des fichiers sources.
  - `affichage.c` : Contient les fonctions dédiées à l'interface utilisateur en ligne de commande (affichage de la grille et score).
  - `fichier.c` : Gère les fonctions de lecture, d'écriture et de sauvegarde des états du jeu pour le système de replay.
  - `ft_itoa.c` et `ft_strmanip.c` : Fonctions utilitaires pour la conversion d'entiers en chaînes de caractères et la manipulation de texte (nécessaires pour le stockage).
  - `main.c` : Point d'entrée principal pour lancer une partie interactive.
  - `replay_main.c` : Programme spécifique permettant de rejouer une partie enregistrée à partir d'un fichier de sauvegarde.
- **Fichier** `Makefile` : Permet une compilation automatisée et simplifiée du projet.
- **Fichier** `README.md` : Documentation simplifiée du projet.
- **Fichier** `.gitignore` : Utilisé pour exclure les fichiers temporaires et les exécutables du dépôt GitLab.

2 Description détaillée du contenu du dossier `srcs`

2.1 Fichier jeu.c

2.1.1 createGrille

- Allouer dynamiquement en mémoire une grille carrée de dimension `TAILLE`.

2.1.2 liberGrille

- **Description :** Parcourt chaque ligne pour libérer les tableaux d'entiers, puis libère le pointeur principal.

2.1.3 ajouterTuile

- **Description :** Calcule le nombre de cases vides, choisit un index au hasard parmi elles, puis place un 4 (selon la probabilité `AVOIRQ`) ou un 2.

2.1.4 calcul

- **Objectif :** Gérer le tassement et la somme des nombres.
- **Paramètres :**
  - `int ligne[TAILLE]` : un tableau représentant une ligne ou colonne extraite.
  - `int *score` : pointeur vers le score global pour le mettre à jour en cas de fusion.
- **Retour :** `1` si un mouvement ou une fusion a eu lieu, `0` sinon.

— **Description :** Effectue trois étapes : déplace les nombres vers l'extrémité (tassement), fusionne les nombres identiques adjacents, puis tasse à nouveau pour combler les vides créés par les fusions.

2.1.5 movGauche, movDroite, enHaut, enBas

— **Objectif :** Exécuter le mouvement complet sur la grille selon la direction choisie.

— **Paramètres :** `int **g`, `int *score`.

— **Description :** Extrait chaque ligne (ou colonne) de la grille dans un tableau temporaire, appelle la fonction `calcul`, puis réinjecte le résultat dans la grille d'origine. Pour les mouvements "droite" ou "bas", la ligne/colonne est traitée de manière inversée.

2.1.6 estTermine

— **Description :** Vérifie d'abord s'il reste des cases vides. Si la grille est pleine, elle vérifie si deux cases adjacentes (horizontalement ou verticalement) possèdent la même valeur, autorisant ainsi un dernier mouvement.

2.1.7 gagner

— Détecter si la condition de victoire (2048) est atteinte.

2.2 Fichier fichier.c

2.2.1 saveStateToFile

— **Objectif :** Enregistrer l'état actuel de la grille et du score dans un fichier texte.

— **Retour :** `1` en cas de succès, `-1` si le fichier ne peut être ouvert.

— **Description du fonctionnement :** La fonction ouvre le fichier en mode "append" (`a`) dans le dossier `savefiles/`. Elle écrit chaque valeur de la grille séparée par le caractère `:`. À la fin de la grille, elle insère le caractère `$` suivi du score, et termine la ligne par un `;`. Cela permet de stocker l'historique complet d'une partie mouvement par mouvement.

2.2.2 initSaveFile

— **Retour :** Une chaîne de caractères contenant le nom du fichier (ex : `Mon_Dec_22_18:00:00_2025.txt`).

— **Description du fonctionnement :** Utilise la bibliothèque `time.h` pour récupérer l'heure système. Elle remplace les espaces par des tirets bas (`_`) pour éviter les erreurs de chemin de fichier et ajoute l'extension `.txt`.

2.2.3 outputLine

— **Objectif :** Décoder une ligne de sauvegarde et l'afficher graphiquement.

— **Paramètres :** `char *line` : une ligne extraite du fichier de sauvegarde.

— **Description du fonctionnement :** Elle alloue une grille temporaire, parcourt la chaîne de caractères pour extraire les entiers situés entre les séparateurs : et les place dans la matrice. Une fois la ligne décodée, elle appelle `affichageGrille` pour montrer cet état précis au joueur.

2.2.4 replayGame

— **Objectif :** Rejouer l'intégralité d'une partie enregistrée.

— **Description du fonctionnement :** Lit le fichier caractère par caractère pour reconstruire chaque ligne de mouvement (jusqu'au `;`). Pour chaque ligne identifiée, elle appelle `outputLine` pour afficher l'évolution de la partie.

2.2.5 lineToGrille

— **Objectif :** Convertir une chaîne de caractères de sauvegarde en une structure de matrice `int **`.

— **Description :** Similaire à `outputLine`, mais au lieu d'afficher, elle retourne la grille et met à jour la variable score via son pointeur.

2.2.6 recupGrilleEnCours

— **Objectif :** Récupérer le dernier état valide d'une partie (utile pour reprendre une partie interrompue).

— **Retour :** La dernière grille enregistrée dans le fichier.

— **Description du fonctionnement :** Parcourt tout le fichier de sauvegarde ligne par ligne et ne conserve en mémoire que la toute dernière grille décodée avant la fin du fichier (`EOF`).

2.3 Fichier ft_itoa.c :

2.3.1 ft_taille

— **Objectif :** Calculer le nombre de caractères nécessaires pour représenter un entier sous forme de chaîne.

— **Description :** Parcourt le nombre en le divisant par 10 de manière itérative jusqu'à atteindre zéro, en comptant chaque itération. Gère le cas particulier du chiffre 0.

2.3.2 fill

— **Description :** Extrait chaque chiffre du nombre à l'aide de l'opérateur modulo (`% 10`), convertit le résultat en caractère ASCII (en ajoutant 48, soit le code de '0') et remplit la chaîne de droite à gauche.

2.3.3 ft_itoa

— **Objectif :** Convertir un entier (`int`) en une chaîne de caractères allouée dynamiquement (`char *`).

— **Description :** La fonction calcule la taille requise, alloue l'espace avec `malloc` (en prévoyant de la place pour le signe '-' si nécessaire et le caractère nul `'\0'`), puis utilise la fonction `fill` pour générer le contenu textuel.

2.3.4 ft_atoi

— **Objectif :** Convertir une chaîne de caractères représentant un nombre en un entier de type `int`.

— **Description du fonctionnement :**

1. Ignore les espaces blancs initiaux.
2. Gère la présence éventuelle d'un signe '+' ou '-'.

3. Parcourt les caractères numériques et reconstruit l'entier en multipliant le résultat précédent par 10 et en ajoutant la valeur du nouveau chiffre.

2.4 Fichier ft_strmanip.c :

2.4.1 ft_strlen

— **Objectif :** Calculer la longueur d'une chaîne de caractères.

— **Description :** Parcourt la chaîne à l'aide d'un compteur jusqu'à rencontrer le caractère de fin de chaîne `'\0'`.

2.4.2 ft_strappend

— **Description :** Alloue un nouvel espace mémoire de taille `longueur + 2` (pour le nouveau caractère et le `'\0'`). Elle copie l'ancienne chaîne, ajoute le caractère à la suite et ferme la chaîne. Cette fonction est notamment utilisée lors de la lecture des fichiers de sauvegarde caractère par caractère.

2.4.3 ft_strjoin

— **Description du fonctionnement :** Alloue un bloc mémoire suffisant pour contenir les deux chaînes. Elle copie d'abord `s1`, puis concatène `s2` juste après. Elle est utilisée pour construire les chemins de fichiers, par exemple pour ajouter le dossier `"savefiles/"` devant le nom d'un fichier.

2.4.4 ft_str_replace

— **Description :** Parcourt la chaîne de bout en bout. Chaque fois qu'elle trouve le caractère `bef`, elle le remplace directement dans la mémoire par `aft`. Cette fonction est utilisée pour "nettoyer" les noms de fichiers (ex : remplacer les espaces de la date par des underscores).

2.5 Fichier affichage.c :

2.5.1 tailleInt

— **Objectif :** Déterminer le nombre de chiffres composant un entier.

— **Description :** Similaire à `ft_taille`, cette fonction permet de calculer dynamiquement l'espace nécessaire pour chaque cellule afin d'ajuster l'alignement de la grille.

2.5.2 affichageGrille

— **Objectif :** Produire un rendu visuel propre et aligné du plateau de jeu.

— **Fonctionnement :**

1. Génère des bordures horizontales (-) et Sépare les cellules par des barres verticales pour un aspect tableau.

2.6 Fichier replay_main.c

2.6.1 replayMode

— **Fonction :** Restitue chronologiquement les états d'une partie.

— **Logique :** Lit le fichier caractère par caractère, reconstruit la grille jusqu'au marqueur `$` et l'affiche. Attend la touche `n` (next) pour passer à l'étape suivante.

2.6.2 main

— **Rôle :** Point d'entrée du mode spectateur. Initialise le chargement via `argv[1]` et pilote l'interaction utilisateur (suivant/quitter).

2.7 Fichier main.c

2.7.1 main (Mode de jeu interactif)

— **Objectif :** Piloter le cycle de vie complet d'une partie.

— **Fonctionnement :**

1. **Initialisation :** Alloue la grille et initialise l'aléatoire (`srand`).
2. **Gestion des modes :** Charge une partie existante via `argv` (mode continu) ou crée une session neuve avec un fichier daté (mode nouveau).
3. **Boucle principale :** Rafraîchit l'affichage, capture les commandes (`g, d, h, b`), traite les mouvements et auto-sauvegarde chaque état valide.
4. **Conclusion :** Détecte la victoire ou l'échec, propose un replay immédiat et libère toute la mémoire allouée.

3 le Makefile

3.1 Configuration technique

— **Variables :** Définit le compilateur (`gcc`), les noms des binaires (`2048` et `replay_mode`) et l'organisation des sources dans `srcs/`.

— **Compilation par objets :** Transforme les fichiers `.c` en `.o` avant le liage final pour optimiser le temps de compilation.

— **Interface :** Utilise des codes couleurs ANSI (`GREEN`, `RED`, etc.) pour un retour visuel clair dans le terminal.

3.2 Commandes principales

— `all` / `replay` **:** Compilent respectivement le jeu et le mode relecture.

— `run` **:** Compile et lance le programme en une seule commande.

— `clean` / `fclean` **:** Suppriment les fichiers temporaires (`.o`) et les exécutables.

— `re` **:** Effectue un cycle de nettoyage complet et de recompilation.

— `resetsavefiles` **:** Réinitialise le dossier des sauvegardes.

4 Fin

4.1 Exécuter

Enfin, pour exécuter, il faut rentrer la commande :

```
make
./2048
```